Project Synopsis

A concise overview of the proposed Database Management System (DBMS) project.

1. General Information

A) Project Title: Campus Complaint & Grievance Tracker — A Trigger-Driven Complaint Lifecycle System

B) Team Members:

- Member 1: Chirag Nikant Simepurushkar |24B-CO-015 | simepurushkarchirag84@gmail.com
- Member 2: Raksham R Dessai| 24B-CO-019 | rakshamdessai2006@gmail.com
- Member 3: Aayushi Rajesh Prabhu | 24B-CO-001 | aayushiprabhu99@gmail.com

2. Project Overview & Problem Statement

- **Introduction:** This project is built for a university/college campus, to track and manage complaints raised by students and staff across all departments — hostel, mess, academics, infrastructure/maintenance, IT, and other campus services.
- **Problem Statement:** Campus complaints are currently reported informally, through WhatsApp groups or verbal requests to wardens and staff. This leads to complaints going unrecorded, no visibility into whether anyone is addressing an issue, no accountability once a complaint is raised, and no escalation path when the responsible staff member does not act in time. There is also no consolidated record that lets the administration see which departments or categories generate the most complaints, or how long issues typically take to resolve.
- **Proposed Solution:** We treat every grievance as a row in a well-designed relational database with an enforced complaint lifecycle, automatic escalation, and role-based visibility. Status transitions, escalation, and audit logging are implemented at the database layer itself using triggers, scheduled events, and views — rather than relying on application code or manual follow-up — so the system remains reliable and consistent regardless of which frontend or client accesses it.

3. Scope and Objectives

- **Core goals:** Design a normalized (3NF/BCNF) relational schema for a multi-department campus complaint system; enforce the complaint lifecycle as a database-level state machine using triggers; implement automatic escalation of unresolved complaints; provide role-based views for Student, Staff, Warden, and Admin; and demonstrate query optimization through indexing and execution-plan comparison.
- **In scope:** Complaint registration and categorization across departments (Hostel, Mess, Academics, Infrastructure, IT, and other categories as needed); a status lifecycle of Open → Assigned → In-Progress → Resolved/Escalated with invalid-transition rejection; automatic escalation after a configurable number of unresolved days; a complete status-change audit trail; and role-based dashboards showing open counts, category-wise trends, and average resolution time.
- **Out of scope:** Real-time push notifications (SMS/email alerts), payment processing, live GPS or camera-based tracking of any kind, native mobile apps (a responsive web frontend is sufficient), and integration with external campus ID or security systems.

4. System Functionalities & User Roles

The system identifies four user roles, each restricted to a role-specific database view rather than direct table access.

- **Student:** Can file a new complaint under any category, view the status and history of only their own complaints, and track escalation progress.
- **Staff:** Can view complaints assigned to them, update status (subject to state-machine validation), and add resolution notes.
- **Warden:** Can view all complaints raised within their hostel, assign/reassign staff, and monitor escalation status hostel-wide.

- **Admin:** Can view all complaints across every department, monitor average resolution time and category-wise trends, and manage department, category, and staff records.

5. Technology Stack

- **Frontend / UI:** React (or plain HTML/CSS/JS) for Student, Warden, and Admin dashboards
- **Backend / Server:** Node.js/Express or Python/Flask — kept intentionally thin, since core logic (state machine, escalation, audit trail) lives in the database layer
- **Database Engine:** MySQL or PostgreSQL (for triggers, scheduled events, and views); MongoDB optionally used for flexible-schema data such as complaint attachments or comment threads